CSCI6730 – Operating Systems

Project #2: Concurrency Bug Investigation

Due date: 11:59pm, 10/17/2024

Description

In this assignment, you will be provided with four different applications, each containing at least one concurrency bug, such as a race condition, deadlock, or live lock. Along with the applications, you will also receive a bug report for each program, detailing initial observations related to the bugs. Your task is to further investigate, identify, and resolve these bugs with minimal changes to the existing codebase.

Preparation

Download the project file (OS_Proj2.tar.gz) from ELC. Then, transfer it to the Odin server using any SSH or SFTP program.

- Once you have the tarball in odin server, you can extract it with the following command:

  ```
  % tar xzvf OS_Proj2.tar.gz
  ```

- You will find four subdirectories, each containing a separate program. The corresponding bug reports are available at the end of this project description.
- You may use any Linux machine for this project, including Odin or vcf0~3. However, make sure your fixed code compiles and runs successfully on Odin, as this is where it will be evaluated.

Project Objectives

1. **Identify the Problem:** Analyze the provided applications and bug reports to identify the concurrency bug(s) present in each.
2. **Reproduce the Bug:** Use the information from the bug report and develop a reproducible test case that reliably demonstrates the bug in action.
3. **Root Cause Analysis:** Investigate and pinpoint the exact cause of the bug by examining the interactions between threads and shared resources.
4. **Fix the Code:** Propose and implement minimal yet effective changes to fix the concurrency issues without introducing new bugs or significantly altering the program's behavior.

Project Report Requirements

You are expected to submit a detailed project report that covers the following:

1. **Problem Identification:** Provide a clear description of each identified bug, including what kind of concurrency issue it is (race condition, deadlock, etc.), supported by the information from the bug report.
2. **Bug Reproduction:** Document how you reproduced the bug, including any steps, commands, or scripts used to trigger the issue.
3. **Root Cause Analysis:** Explain your investigation process and provide a detailed analysis of the root cause of the bug.
4. **Fix Design:** Describe your proposed fixes, including a rationale for why they resolve the issue. Discuss the minimal changes you applied and how they address the concurrency problem.
5. **Discussion of Fixes:** Include a discussion of why the fix is effective, the potential for any new bugs, and any trade-offs or considerations made during the fix.
6. **Testing and Validation:** Provide evidence (logs, screenshots, or tests) that demonstrate the effectiveness of your fix and confirm that no new bugs were introduced.

Only "pdf" format will be accepted.

Project Report Requirements

Extra credit will be awarded if you identify and resolve additional concurrency bugs that are not covered in the provided bug reports. Make sure to include any additional bugs in your report, with clear documentation of the reproduction, root cause analysis, and fix.

Grading Criteria

Your project will be graded based on:

- The correctness and completeness of bug identification.
- The effectiveness and minimalism of your bug fixes.
- The quality and depth of your root cause analysis.
- The clarity and thoroughness of your report.
- The success of your testing and validation process.
- Extra credit will be considered for identifying and fixing additional bugs not mentioned in the original bug reports.

Bug Reports:

1. Worker – Segmentation Fault

I created a simple toy program that simulates a multi-threaded worker system. Each worker thread continuously generates data until it receives a SIGINT signal (triggered by pressing ctrl+c). Once SIGINT is received, the main thread collects all the data and cleans up memory before terminating.

The program works fine in most cases, but I've noticed occasional segmentation faults when sending ctrl+c, as shown below:

```
[kyuhlee@csci-odin worker]$ ./worker
^CMain: caught SIGINT (Ctrl+C). Exiting gracefully.
Segmentation fault (core dumped)
[kyuhlee@csci-odin worker]$
```

Each worker thread uses a global data object, __thread_data, which is shared between that specific worker thread and the main thread. However, the main thread only accesses this shared object after the SIGINT signal is delivered, so I assumed that no lock would be necessary.

Upon further investigation, I found that the segmentation fault occurs at line 51, where the worker threads update the global data object.

To compile and run the code, you can use the following commands:

```
% cd worker
% make
% ./worker
```

2. StringBuffer – Assertion Failed

StringBuffer is a thread-safe class that provides various useful functionalities for strings, including append, erase, insert, and print. I frequently use this class in my program, but I've encountered an "Assertion failed" error when multiple threads access the shared string buffer. I suspect this is a race condition, as the issue occurs non-deterministically.

To reproduce the problem, I created a simple program where one thread continuously generates a String object and appends it to the shared buffer, while another thread erases content from the buffer and then appends new strings.
The screenshot of the failed run with the assertion error is shown below:

```
[kyuhlee@csci-odin stringbuffer]$ ./test
0th try - Thread 1: erase and append: 0
0th try - Thread 2: create and append: 0
100th try - Thread 1: erase and append: 1
100th try - Thread 2: create and append: 1
200th try - Thread 1: erase and append: 2
200th try - Thread 2: create and append: 2
300th try - Thread 1: erase and append: 3
300th try - Thread 2: create and append: 3
400th try - Thread 1: erase and append: 4
400th try - Thread 2: create and append: 4
500th try - Thread 1: erase and append: 5
500th try - Thread 2: create and append: 5
600th try - Thread 1: erase and append: 6
600th try - Thread 2: create and append: 6
700th try - Thread 1: erase and append: 7
700th try - Thread 2: create and append: 7
test: stringbuffer.cpp:54: void StringBuffer::getChars(int, int, char*, int): Assertion `0' failed.
Aborted (core dumped)
```

Although all the methods in the StringBuffer class are fully protected by *mutex_lock*, a race condition still seems to occur, causing the assertion failure.

You can compile and run the test program using the following commands:

```
% cd stringbuffer
% make
% ./test
```

3. Pbzip2 - Segmentation Fault

I encountered a segmentation fault when using Pbzip2 to compress a file. The program occasionally crashes, and I was able to reproduce the issue using GDB. The crash occurs when the program calls “pthread_mutex_unlock”.

Here is the backtrace:

```
         File #: 1 of 1
     Input Name: test.file
    Output Name: test.file.bz2

     Input Size: 2930969 bytes
Compressing data...
[New Thread 0x7ffff6dc0700 (LWP 13419)]
[New Thread 0x7ffff65bf700 (LWP 13420)]
[New Thread 0x7ffff5dbe700 (LWP 13421)]
[New Thread 0x7ffff55bd700 (LWP 13422)]
[New Thread 0x7ffff4dbc700 (LWP 13423)]
[New Thread 0x7ffff45bb700 (LWP 13424)]
[New Thread 0x7ffff3dba700 (LWP 13425)]
[New Thread 0x7ffff35b9700 (LWP 13426)]
[New Thread 0x7ffff2db8700 (LWP 13427)]
[New Thread 0x7ffff25b7700 (LWP 13428)]
[New Thread 0x7ffff1db6700 (LWP 13429)]
[New Thread 0x7ffff15b5700 (LWP 13430)]
[New Thread 0x7ffff0db4700 (LWP 13431)]
[New Thread 0x7ffff05b3700 (LWP 13432)]
[New Thread 0x7fffefdb2700 (LWP 13433)]
[New Thread 0x7fffef5b1700 (LWP 13434)]
[New Thread 0x7fffeedb0700 (LWP 13435)]
[New Thread 0x7fffee5af700 (LWP 13436)]
[New Thread 0x7fffeddae700 (LWP 13437)]
[New Thread 0x7fffed5ad700 (LWP 13438)]
[New Thread 0x7fffecdac700 (LWP 13439)]
[New Thread 0x7fffec5ab700 (LWP 13440)]
[New Thread 0x7fffebdaa700 (LWP 13441)]
[New Thread 0x7fffeb5a9700 (LWP 13442)]
[New Thread 0x7fffeada8700 (LWP 13443)]
[New Thread 0x7fffea5a7700 (LWP 13444)]
[New Thread 0x7fffe9da6700 (LWP 13445)]
[New Thread 0x7fffe95a5700 (LWP 13446)]
[New Thread 0x7fffe8da4700 (LWP 13447)]
[New Thread 0x7fffe85a3700 (LWP 13448)]
[New Thread 0x7fffe7da2700 (LWP 13449)]
[New Thread 0x7fffe75a1700 (LWP 13450)]
[New Thread 0x7fffe6da0700 (LWP 13451)]
[New Thread 0x7fffe659f700 (LWP 13452)]
[New Thread 0x7fffe5d9e700 (LWP 13453)]
[New Thread 0x7fffe559d700 (LWP 13454)]
[New Thread 0x7fffe4d9c700 (LWP 13455)]
    Output Size: 875176 bytes
-------------------------------------------
[Thread 0x7fffe4d9c700 (LWP 13455) exited]

Program received signal SIGSEGV, Segmentation fault.
[Switching to Thread 0x7fffe6da0700 (LWP 13451)]
0x00007ffff7bc9ee0 in pthread_mutex_unlock () from /lib64/libpthread.so.0
Missing separate debuginfos, use: debuginfo-install bzip2-libs-1.0.6-13.el7.x86_64 glibc-2.17-326.el7_9.
3.x86_64 libgcc-4.8.5-44.el7.x86_64 libstdc++-4.8.5-44.el7.x86_64
(gdb) bt
#0  0x00007ffff7bc9ee0 in pthread_mutex_unlock () from /lib64/libpthread.so.0
#1  0x0000000000402c53 in consumer(void*) ()
#2  0x00007ffff7bc6ea5 in start_thread () from /lib64/libpthread.so.0
#3  0x00007ffff6ebfb0d in clone () from /lib64/libc.so.6
(gdb)
```

I suspect this is a race condition, as the crash occurs infrequently. It might be helpful to test by injecting an artificial delay before the pthread_mutex_unlock call, which triggers the segmentation fault.

The following are the commands that I used to compile and run the program:

- `% cd pbzip2`
- `% make`
- `% ./pbzip2 -k -f test.file`

I noticed that the output file (test.file.bz2) is correctly compressed and written, so I suspect the bug occurs at the end of the execution, after the output files have been successfully created.

4. Aget – Corrupted Log

When using Aget to download files from the internet, I occasionally need to stop the download and resume it later. Aget supports this functionality smoothly by maintaining a log that records the download progress. I can simply stop the download with *ctrl+c* and resume it later.

Aget also provides a separate tool, called verifier, to check the integrity of the stopped download. While this process typically works as expected, I've encountered an issue where the log file becomes corrupted. Specifically, the size of the downloaded content and the total bytes written do not match.

For example, in the following scenario:

1. I compile the program using the following command:
   `% cd aget`
   `% make`
2. I started a download using the command:
   `%./aget -n2 http://ftp.us.debian.org/debian-cdimage/12.7.0/i386/iso-cd/debian-edu-12.7.0-i386-netinst.iso -l ./test.download`
3. I stopped the download by pressing ctrl+c.
4. Before resuming the download, I verified the integrity of the log with this command:
   `%./verifier /home/myid/kyuhlee/test.download-ageth.log`
   Note that the full path of the log file is displayed on the screen.

However, the log was corrupted, showing a mismatch between the logged size and the actual downloaded content. See the screenshot below.

```
[kyuhlee@csci-odin aget]$ ./aget -n2 http://ftp.us.debian.org/debian-cdimage/12.7.0/i386/iso
-cd/debian-edu-12.7.0-i386-netinst.iso -l ./test.download
Aget > Attempting to read log file for resuming download job...
Aget > Couldn't find log file for this download, starting a clean job...
Aget > Head-Request Connection established
Aget > Downloading /debian-cdimage/12.7.0/i386/iso-cd/debian-edu-12.7.0-i386-netinst.iso (68
3671552 bytes) from site ftp.us.debian.org(64.50.233.100:80).                    ^C^C c
aught, terminating download job. Please wait...
Download job terminated. Now saving download job... [00% done] Bytes: 15744
--> Logfile is: /home/myid/kyuhlee/test.download-ageth.log, so far 44946 bytes have been tra
nsferred
[kyuhlee@csci-odin aget]$
[kyuhlee@csci-odin aget]$
[kyuhlee@csci-odin aget]$ ./verifier /home/myid/kyuhlee/test.download-ageth.log
/home/myid/kyuhlee/test.download-ageth.log is incorrect and may be corrupted. Deleting the l
og.
Total bytes written: 44946, Size of content: 61330 bytes.
[kyuhlee@csci-odin aget]$
```

Upon investigating the code, it appears that the download process is handled by the **http_get** function in the *Download.c* file, which tracks the total bytes written. When the download is stopped, the **save_log** function in *Resume.c* is called to save the log file. It seems the issue lies somewhere in this log-saving process, leading to occasional corruption.

Submission

1. Prepare a tarball file using the following command
   % cd OS_Proj2
   % tar czvf p2.tar.gz report.pdf agetpbzip2 stringbuffer worker
   (Ensure to include your fixed code in the tarball).
2. Your report must include your 8xx number and name.
3. **Ensure that your fixed code compiles and runs correctly on the odin machine.**
   a. The code will be evaluated on the odin machine.
   b. If your code fails to compile on odin, you will receive no points for the testing and evaluation portion.
4. Submit the tarball (p2.tar.gz) through ELC.